```
      new HashMap<Integer, Queue<Integer>>();
   for (int s : small) {
      Queue<Integer> queue = new LinkedList<Integer>();
      itemLocations.put(s, queue);
   }

   / *Walk through big array, adding the item locations to hash map */
   for (int i = 0; i < big.length; i++) {
      Queue<Integer> queue = itemLocations.get(big[i]);
      if (queue != null) {
         queue.add(i);
      }
   }

   ArrayList<Queue<Integer>> allLocations = new ArrayList<Queue<Integer>>();
   allLocations.addAll(itemLocations.values());
   return allLocations;
}

Range getShortestClosure(ArrayList<Queue<Integer>> lists) {
   PriorityQueue<HeapNode> minHeap = new PriorityQueue<HeapNode>();
   int max = Integer.MIN_VALUE;

   / *Insert min element from each list. */
   for (int i = 0; i < lists.size(); i++) {
      int head = lists.get(i).remove();
      minHeap.add(new HeapNode(head, i));
      max = Math.max(max, head);
   }

   int min = minHeap.peek().locationWithinList;
   int bestRangeMin = min;
   int bestRangeMax = max;

   while (true) {
      / *Remove min node. */
      HeapNode n = minHeap.poll();
      Queue<Integer> list = lists.get(n.listId);

      / *Compare range to best range. */
      min = n.locationWithinList;
      if (max - min < bestRangeMax - bestRangeMin) {
         bestRangeMax = max;
         bestRangeMin = min;
      }

      / *If there are no more elements, then there's no more subsequences and we
       * can break. */
      if (list.size() == 0) {
         break;
      }

      / *Add new head of list to heap. */
      n.locationWithinList = list.remove();
      minHeap.add(n);
      max = Math.max(max, n.locationWithinList);
```

```
68      }
69
70      return new Range(bestRangeMin, bestRangeMax);
71  }
```

We're going through B elements in `getShortestClosure`, and each time pass in the for loop will take `O(log S)` time (the time to insert/remove from the heap). This algorithm will therefore take `O(B log S)` time in the worst case.

17.19 Missing Two: You are given an array with all the numbers from 1 to N appearing exactly once, except for one number that is missing. How can you find the missing number in `O(N)` time and `O(1)` space? What if there were two numbers missing?

pg 189

SOLUTIONS

Let's start with the first part: find a missing number in `O(N)` time and `O(1)` space.

Part 1: Find One Missing Number

We have a very constrained problem here. We can't store all the values (that would take `O(N)` space) and yet, somehow, we need to have a "record" of them such that we can identify the missing number.

This suggests that we need to do some sort of computation with the values. What characteristics does this computation need to have?

- **Unique.** If this computation gives the same result on two arrays (which fit the description in the problem), then those arrays must be equivalent (same missing number). That is, the result of the computation must uniquely correspond to the specific array and missing number.
- **Reversible.** We need some way of getting from the result of the calculation to the missing number.
- **Constant Time:** The calculation can be slow, but it must be constant time per element in the array.
- **Constant Space:** The calculation can require additional memory, but it must be `O(1)` memory.

The "unique" requirement is the most interesting—and the most challenging. What calculations can be performed on a set of numbers such that the missing number will be discoverable?

There are actually a number of possibilities.

We could do something with prime numbers. For example, for each value `x` in the array, we multiply `result` by the xth prime. We would then get some value that is indeed unique (since two different sets of primes can't have the same product).

Is this reversible? Yes. We could take `result` and divide it by each prime number: 2, 3, 5, 7, and so on. When we get a non-integer for the `i`th prime, then we know `i` was missing from our array.

Is it constant time and space, though? Only if we had a way of getting the `i`th prime number in `O(1)` time and `O(1)` space. We don't have that.

What other calculations could we do? We don't even need to do all this prime number stuff. Why not just multiply all the numbers together?

- **Unique?** Yes. Picture `1*2*3*...*n`. Now, imagine crossing off one number. This will give us a different result than if we crossed off any other number.
- **Constant time and space?** Yes.

- **Reversible?** Let's think about this. If we compare what our product is to what it would have been without a number removed, can we find the missing number? Sure. We just divide `full_product` by `actual_product`. This will tell us which number was missing from `actual_product`.

There's just one issue: this product is really, really, really big. *If* n *is* 20, the product *will* be somewhere around 2,000,000,000,000,000,000.

We can still approach it this way, but we'll need to use the `BigInteger` class.

```
1  int missingOne(int[] array) {
2    BigInteger fullProduct = productToN(array.length + 1);
3
4    BigInteger actualProduct = new BigInteger("1");
5    for (int i = 0; i < array.length; i++) {
6      BigInteger value = new BigInteger(array[i] + "");
7      actualProduct = actualProduct.multiply(value);
8    }
9
10   BigInteger missingNumber = fullProduct.divide(actualProduct);
11   return Integer.parseInt(missingNumber.toString());
12 }
13
14 BigInteger productToN(int n) {
15   BigInteger fullProduct = new BigInteger("1");
16   for (int i = 2; i <= n; i++) {
17     fullProduct = fullProduct.multiply(new BigInteger(i + ""));
18   }
19   return fullProduct;
20 }
```

There's no need for all of this, though. We can use the sum instead. It too will be unique.

Doing the sum has another benefit: there is already a closed form expression to compute the sum of numbers between 1 and n. This is $\frac{n(n+1)}{2}$.

> Most candidates probably won't remember the expression for the sum of numbers between 1 and n, and that's okay. Your interviewer might, however, ask you to derive it. Here's how to think about that: you can pair up the low and high values in the sequence of `0 + 1 + 2 + 3 + ... + n` to get: `(0, n) + (1, n-1) + (2, n-3)`, and so on. Each of those pairs has a sum of n and there are $\frac{n+1}{2}$ pairs. But what if n is even, such that $\frac{n+1}{2}$ is not an integer? In this case, pair up low and high values to get $n/2$ pairs with sum n+1. Either way, the math works out to $\frac{n(n+1)}{2}$.

Switching to a sum will delay the overflow issue substantially, but it won't wholly prevent it. You should discuss the issue with your interviewer to see how he/she would like you to handle it. Just mentioning it is plenty sufficient for many interviewers.

Part 2: Find Two Missing Numbers

This is substantially more difficult. Let's start with what our earlier approaches will tell us when we have two missing numbers.

- Sum: Using this approach will give us the sum of the two values that are missing.
- Product: Using this approach will give us the product of the two values that are missing.

Unfortunately, knowing the sum isn't enough. If, for example, the sum is 10, that could correspond to (1, 9), (2, 8), and a handful of other pairs. The same could be said for the product.

We're again at the same point we were in the first part of the problem. We need a calculation that can be applied such that the result is unique across all potential pairs of missing numbers.

Perhaps there is such a calculation (the prime one would work, but it's not constant time), but your interviewer probably doesn't expect you to know such math.

What else can we do? Let's go back to what we can do. We can get `x + y` and we can also get x * y. Each result leaves us with a number of possibilities. But using both of them narrows it down to the specific numbers.

```
x + y = sum     -> y = sum - x
x * y = product -> x(sum - x) = product
                  x*sum - x² = product
                  x*sum - x² - product = 0
                  -x² + x*sum - product = 0
```

At this point, we can apply the quadratic formula to solve for x. Once we have x, we can then compute y.

There are actually a number of other calculations you can perform. In fact, almost any other calculation (other than "linear" calculations) will give us values for x and y.

For this part, let's use a different calculation. Instead of using the product of `1 * 2 * ... * n`, we can use the sum of the squares: $1^2 + 2^2 + \ldots + n^2$. This will make the `BigInteger` usage a little less critical, as the code will at least run on small values of n. We can discuss with our interviewer whether or not this is important.

```
x + y = s       -> y = s - x
x² + y² = t     -> x² + (s-x)² = t
                   2x² - 2sx + s²-t = 0
```

Recall the quadratic formula:

```
x = [-b +- sqrt(b² - 4ac)] / 2a
```

where, in this case:

```
a = 2
b = -2s
c = s²-t
```

Implementing this is now somewhat straightforward.

```
1  int[] missingTwo(int[] array) {
2    int max_value = array.length + 2;
3    int rem_square = squareSumToN(max_value, 2);
4    int rem_one = max_value * (max_value + 1) / 2;
5
6    for (int i = 0; i < array.length; i++) {
7      rem_square -= array[i] * array[i];
8      rem_one -= array[i];
9    }
10
11   return solveEquation(rem_one, rem_square);
12 }
13
14 int squareSumToN(int n, int power) {
15   int sum = 0;
16   for (int i = 1; i <= n; i++) {
17     sum += (int) Math.pow(i, power);
18   }
19   return sum;
20 }
```

```
21
22 int[] solveEquation(int r1, int r2) {
23    /* ax^2 + bx + c
24     * -->
25     * x = [-b +- sqrt(b^2 - 4ac)] / 2a
26     * In this case, it has to be a + not a - */
27    int a = 2;
28    int b = -2 * r1;
29    int c = r1 * r1 - r2;
30
31    double part1 = -1 * b;
32    double part2 = Math.sqrt(b*b - 4 * a * c);
33    double part3 = 2 * a;
34
35    int solutionX = (int) ((part1 + part2) / part3);
36    int solutionY = r1 - solutionX;
37
38    int[] solution = {solutionX, solutionY};
39    return solution;
40 }
```

You might notice that the quadratic formula usually gives us two answers (see the + or - part), yet in our code, we only use the (+) result. We never checked the (-) answer. Why is that?

The existence of the "alternate" solution doesn't mean that one is the correct solution and one is "fake." It means that there are exactly two values for x which will correctly fulfill our equation: $2x^2 - 2sx + (s^2-t) = 0$.

That's true. There are. What's the other one? The other value is y!

If this doesn't immediately make sense to you, remember that x and y are interchangeable. Had we solved for y earlier instead of x, we would have wound up with an identical equation: $2y^2 - 2sy + (s^2-t) = 0$. So of course y could fulfill x's equation and x could fulfill y's equation. They have the exact same equation. Since x and y are both solutions to equations that look like $2[something]^2 - 2s[something] + s^2-t = 0$, then the other something that fulfills that equation must be y.

Still not convinced? Okay, we can do some math. Let's say we took the alternate value for x: $[-b - sqrt(b^2 - 4ac)] / 2a$. What's y?

$$\begin{aligned} x + y &= r_1 \\ y &= r_1 - x \\ &= r_1 - [-b - sqrt(b^2 - 4ac)]/2a \\ &= [2a*r_1 + b + sqrt(b^2 - 4ac)]/2a \end{aligned}$$

Partially plug in values for a and b, but keep the rest of the equation as-is:

$$\begin{aligned} &= [2(2)*r_1 + (-2r_1) + sqrt(b^2 - 4ac)]/2a \\ &= [2r_1 + sqrt(b^2 - 4ac)]/2a \end{aligned}$$

Recall that $b = -2r_1$. Now, we wind up with this equation:

$$= [-b + sqrt(b^2 - 4ac)]/2a$$

Therefore, if we use x = (part1 + part2) / part3, then we'll get (part1 - part2) / part3 for the value for y.

We don't care which one we call x and which one we call y, so we can use either one. It'll work out the same in the end.

17.20 *Continuous Median:* Numbers are randomly generated and passed to a method. Write a program to find and maintain the median value as new values are generated.

pg 189

SOLUTIONS

One solution is to use two priority heaps: a max heap for the values below the median, and a min heap for the values above the median. This will divide the elements roughly in half, with the middle two elements as the top of the two heaps. This makes it trivial to find the median.

What do we mean by "roughly in half," though? "Roughly" means that, if we have an odd number of values, one heap will have an extra value. Observe that the following is true:

- If `maxHeap.size() > minHeap.size()`, `maxHeap.top()` will be the median.
- If `maxHeap.size() == minHeap.size()`, then the average of `maxHeap.top()` and `minHeap.top()` will be the median.

By the way in which we rebalance the heaps, we will ensure that it is always `maxHeap` with extra element.

The algorithm works as follows. When a new value arrives, it is placed in the `maxHeap` if the value is less than or equal to the median, otherwise it is placed into the `minHeap`. The heap sizes can be equal, or the `maxHeap` may have one extra element. This constraint can easily be restored by shifting an element from one heap to the other. The median is available in constant time, by looking at the top element(s). Updates take `O(log(n))` time.

```
1  Comparator<Integer> maxHeapComparator, minHeapComparator;
2  PriorityQueue<Integer> maxHeap, minHeap;
3
4  void addNewNumber(int randomNumber) {
5    /* Note: addNewNumber maintains a condition that
6     * maxHeap.size() >= minHeap.size() */
7    if (maxHeap.size() == minHeap.size()) {
8      if ((minHeap.peek() != null) &&
9          randomNumber > minHeap.peek()) {
10       maxHeap.offer(minHeap.poll());
11       minHeap.offer(randomNumber);
12     } else {
13       maxHeap.offer(randomNumber);
14     }
15   } else {
16     if(randomNumber < maxHeap.peek()) {
17       minHeap.offer(maxHeap.poll());
18       maxHeap.offer(randomNumber);
19     }
20     else {
21       minHeap.offer(randomNumber);
22     }
23   }
24 }
25
26 double getMedian() {
27   /* maxHeap is always at least as big as minHeap. So if maxHeap is empty, then
28    * minHeap is also. */
29   if (maxHeap.isEmpty()) {
30     return 0;
31   }
```

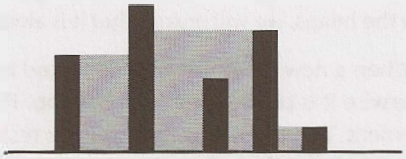
```
32      if (maxHeap.size() == minHeap.size()) {
33          return ((double)minHeap.peek()+(double)maxHeap.peek()) / 2;
34      } else {
35          /* If maxHeap and minHeap are of different sizes, then maxHeap must have one
36           * extra element. Return maxHeap's top element.*/
37          return maxHeap.peek();
38      }
39  }
```

17.21 Volume of Histogram: Imagine a histogram (bar graph). Design an algorithm to compute the volume of water it could hold if someone poured water across the top. You can assume that each histogram bar has width 1.

EXAMPLE

Input: {0, 0, 4, 0, 0, 6, 0, 0, 3, 0, 5, 0, 1, 0, 0, 0}

(Black bars are the histogram. Gray is water.)

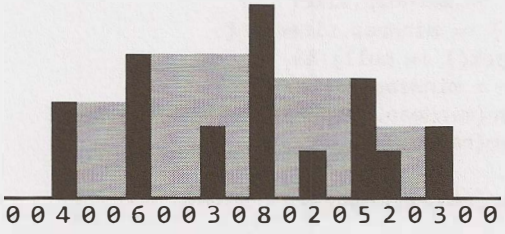


Output: 26

pg 189

SOLUTION

This is a difficult problem, so let's come up with a good example to help us solve it.



We should study this example to see what we can learn from it. What exactly dictates how big those gray areas are?

Solution #1

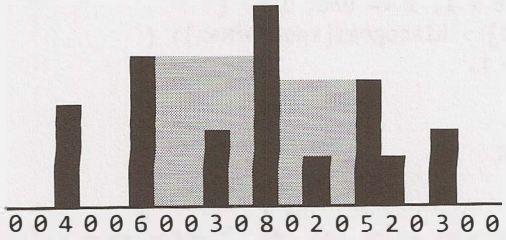
Let's look at the tallest bar, which has size 8. What role does that bar play? It plays an important role for being the highest, but it actually wouldn't matter if that bar instead had height 100. It wouldn't affect the volume.

The tallest bar forms a barrier for water on its left and right. But the volume of water is actually controlled by the next highest bar on the left and right.

- **Water on immediate left of tallest bar:** The next tallest bar on the left has height 6. We can fill up the area in between with water, but we have to deduct the height of each histogram between the tallest and next tallest. This gives a volume on the immediate left of: `(6-0) + (6-0) + (6-3) + (6-0) = 21`.
- **Water on immediate right of tallest bar:** The next tallest bar on the right has height 5. We can now

compute the volume: (5-0) + (5-2) + (5-0) = 13.

This just tells us part of the volume.



What about the rest?

We have essentially two subgraphs, one on the left and one on the right. To find the volume there, we repeat a very similar process.

1. Find the max. (Actually, this is given to us. The highest on the left subgraph is the right border (6) and the highest on the right subgraph is the left border (5).)
2. Find the second tallest in each subgraph. In the left subgraph, this is 4. In the right subgraph, this is 3.
3. Compute the volume between the tallest and the second tallest.
4. Recurse on the edge of the graph.

The code below implements this algorithm.

```
1   int computeHistogramVolume(int[] histogram) {
2     int start = 0;
3     int end = histogram.length - 1;
4
5     int max = findIndexOfMax(histogram, start, end);
6     int leftVolume =  subgraphVolume(histogram, start, max, true);
7     int rightVolume = subgraphVolume(histogram, max, end, false);
8
9     return leftVolume + rightVolume;
10  }
11
12  /* Compute the volume of a subgraph of the histogram. One max is at either start
13   * or end (depending on isLeft). Find second tallest, then compute volume between
14   * tallest and second tallest. Then compute volume of subgraph. */
15  int subgraphVolume(int[] histogram, int start, int end, boolean isLeft) {
16    if (start >= end) return 0;
17    int sum = 0;
18    if (isLeft) {
19      int max = findIndexOfMax(histogram, start, end - 1);
20      sum += borderedVolume(histogram, max, end);
21      sum += subgraphVolume(histogram, start, max, isLeft);
22    } else {
23      int max = findIndexOfMax(histogram, start + 1, end);
24      sum += borderedVolume(histogram, start, max);
25      sum += subgraphVolume(histogram, max, end, isLeft);
26    }
27
28    return sum;
29  }
30
```

```
31 /* Find tallest bar in histogram between start and end. */
32 int findIndexOfMax(int[] histogram, int start, int end) {
33    int indexOfMax = start;
34    for (int i = start + 1; i <= end; i++) {
35       if (histogram[i] > histogram[indexOfMax]) {
36          indexOfMax = i;
37       }
38    }
39    return indexOfMax;
40 }
41
42 /* Compute volume between start and end. Assumes that tallest bar is at start and
43  * second tallest is at end. */
44 int borderedVolume(int[] histogram, int start, int end) {
45    if (start >= end) return 0;
46
47    int min = Math.min(histogram[start], histogram[end]);
48    int sum = 0;
49    for (int i = start + 1; i < end; i++) {
50       sum += min - histogram[i];
51    }
52    return sum;
53 }
```

This algorithm takes $O(N^2)$ time in the worst case, where N is the number of bars in the histogram. This is because we have to repeatedly scan the histogram to find the max height.

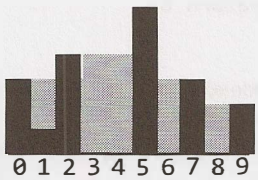
Solution #2 (Optimized)

To optimize the previous algorithm, let's think about the exact cause of the inefficiency of the prior algorithm. The root cause is the perpetual calls to `findIndexOfMax`. This suggests that it should be our focus for optimizing.

One thing we should notice is that we don't pass in arbitrary ranges into the `findIndexOfMax` function. It's actually always finding the max from one point to an edge (either the right edge or the left edge). Is there a quicker way we could know what the max height is from a given point to each edge?

Yes. We could precompute this information in $O(N)$ time.

In two sweeps through the histogram (one moving right to left and the other moving left to right), we can create a table that tells us, from any index `i`, the location of the max index on the right and the max index on the left.



```
INDEX: 0 1 2 3 4 5 6 7 8 9
HEIGHT: 3 1 4 0 0 6 0 3 0 2
INDEX LEFT MAX: 0 0 2 2 2 5 5 5 5 5
INDEX RIGHT MAX: 5 5 5 5 5 5 7 7 9 9
```

The rest of the algorithm precedes essentially the same way.

We've chosen to use a HistogramData object to store this extra information, but we could also use a two-dimensional array.

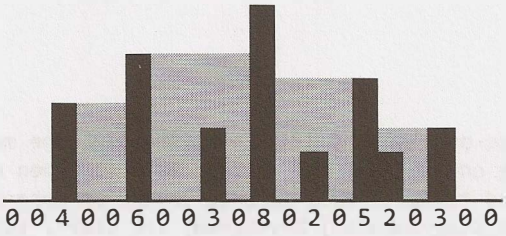
```
int computeHistogramVolume(int[] histogram) {
  int start = 0;
  int end = histogram.length - 1;

  HistogramData[] data = createHistogramData(histogram);

  int max = data[0].getRightMaxIndex(); // Get overall max
  int leftVolume =  subgraphVolume(data, start, max, true);
  int rightVolume = subgraphVolume(data, max, end, false);

  return leftVolume + rightVolume;
}

HistogramData[] createHistogramData(int[] histo) {
  HistogramData[] histogram = new HistogramData[histo.length];
  for (int i = 0; i < histo.length; i++) {
    histogram[i] = new HistogramData(histo[i]);
  }

  /* Set left max index. */
  int maxIndex = 0;
  for (int i = 0; i < histo.length; i++) {
    if (histo[maxIndex] < histo[i]) {
      maxIndex = i;
    }
    histogram[i].setLeftMaxIndex(maxIndex);
  }

  /* Set right max index. */
  maxIndex = histogram.length - 1;
  for (int i = histogram.length - 1; i >= 0; i--) {
    if (histo[maxIndex] < histo[i]) {
      maxIndex = i;
    }
    histogram[i].setRightMaxIndex(maxIndex);
  }

  return histogram;
}

/* Compute the volume of a subgraph of the histogram. One max is at either start
 * or end (depending on isLeft). Find second tallest, then compute volume between
 * tallest and second tallest. Then compute volume of subgraph. */
int subgraphVolume(HistogramData[] histogram, int start, int end,
                   boolean isLeft) {
  if (start >= end) return 0;
  int sum = 0;
  if (isLeft) {
    int max = histogram[end - 1].getLeftMaxIndex();
    sum += borderedVolume(histogram, max, end);
    sum += subgraphVolume(histogram, start, max, isLeft);
  } else {
    int max = histogram[start + 1].getRightMaxIndex();
    sum += borderedVolume(histogram, start, max);
```

```
55        sum += subgraphVolume(histogram, max, end, isLeft);
56      }
57
58      return sum;
59    }
60
61    /* Compute volume between start and end. Assumes that tallest bar is at start and
62     * second tallest is at end. */
63    int borderedVolume(HistogramData[] data, int start, int end) {
64      if (start >= end) return 0;
65
66      int min = Math.min(data[start].getHeight(), data[end].getHeight());
67      int sum = 0;
68      for (int i = start + 1; i < end; i++) {
69        sum += min - data[i].getHeight();
70      }
71      return sum;
72    }
73
74    public class HistogramData {
75      private int height;
76      private int leftMaxIndex = -1;
77      private int rightMaxIndex = -1;
78
79      public HistogramData(int v) { height = v; }
80      public int getHeight() { return height; }
81      public int getLeftMaxIndex() { return leftMaxIndex; }
82      public void setLeftMaxIndex(int idx) { leftMaxIndex = idx; };
83      public int getRightMaxIndex() { return rightMaxIndex; }
84      public void setRightMaxIndex(int idx) { rightMaxIndex = idx; };
85    }
```

This algorithm takes O(N) time. Since we have to look at every bar, we cannot do better than this.

Solution #3 (Optimized & Simplified)

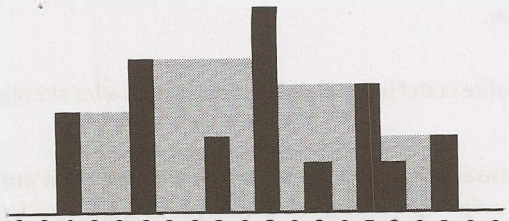
While we can't make the solution faster in terms of big O, we can make it much, much simpler. Let's look at an example again in light of what we've just learned about potential algorithms.



As we've seen, the volume of water in a particular area is determined by the tallest bar to the left and to the right (specifically, by the shorter of the two tallest bars on the left and the tallest bar on the right). For example, water fills in the area between the bar with height 6 and the bar with height 8, up to a height of 6. It's the second tallest, therefore, that determines the height.

The total volume of water is the volume of water above each histogram bar. Can we efficiently compute how much water is above each histogram bar?

Yes. In Solution #2, we were able to precompute the height of the tallest bar on the left and right of each index. The minimums of these will indicate the "water level" at a bar. The difference between the water level and the height of this bar will be the volume of water.



```
HEIGHT: 0 0 4 0 0 6 0 0 3 0 8 0 2 0 5 2 0 3 0 0
LEFT MAX: 0 0 4 4 4 6 6 6 6 6 8 8 8 8 8 8 8 8 8
RIGHT MAX: 8 8 8 8 8 8 8 8 8 8 5 5 5 5 3 3 3 0 0
MIN: 0 0 4 4 4 6 6 6 6 6 8 5 5 5 5 3 3 3 0 0
DELTA: 0 0 0 4 4 0 6 6 3 6 0 5 3 5 0 1 3 0 0 0
```

Our algorithm now runs in a few simple steps:

1. Sweep left to right, tracking the max height you've seen and setting left max.
2. Sweep right to left, tracking the max height you've seen and setting right max.
3. Sweep across the histogram, computing the minimum of the left max and right max for each index.
4. Sweep across the histogram, computing the delta between each minimum and the bar. Sum these deltas.

In the actual implementation, we don't need to keep so much data around. Steps 2, 3, and 4 can be merged into the same sweep. First, compute the left maxes in one sweep. Then sweep through in reverse, tracking the right max as you go. At each element, calculate the min of the left and right max and then the delta between that (the "min of maxes") and the bar height. Add this to the sum.

```
1  /* Go through each bar and compute the volume of water above it.
2   * Volume of water at a bar =
3   *   height - min(tallest bar on left, tallest bar on right)
4   *   [where above equation is positive]
5   * Compute the left max in the first sweep, then sweep again to compute the right
6   * max, minimum of the bar heights, and the delta. */
7  int computeHistogramVolume(int[] histo) {
8    /* Get left max */
9    int[] leftMaxes = new int[histo.length];
10   int leftMax = histo[0];
11   for (int i = 0; i < histo.length; i++) {
12     leftMax = Math.max(leftMax, histo[i]);
13     leftMaxes[i] = leftMax;
14   }
15
16   int sum = 0;
17
18   /* Get right max */
19   int rightMax = histo[histo.length - 1];
20   for (int i = histo.length - 1; i >= 0; i--) {
21     rightMax = Math.max(rightMax, histo[i]);
22     int secondTallest = Math.min(rightMax, leftMaxes[i]);
23
24     /* If there are taller things on the left and right side, then there is water
25      * above this bar. Compute the volume and add to the sum. */
26       if (secondTallest > histo[i]) {
```

```
            sum += secondTallest - histo[i];
        }
    }

    return sum;
}
```

Yes, this really is the entire code! It is still O(N) time, but it's a lot simpler to read and write.

17.22 Word Transformer: Given two words of equal length that are in a dictionary, write a method to transform one word into another word by changing only one letter at a time. The new word you get in each step must be in the dictionary.

EXAMPLE

Input: DAMP, LIKE

Output: DAMP -> LAMP -> LIMP -> LIME -> LIKE

pg 189

SOLUTION

Let's start with a naive solution and then work our way to a more optimal solution.

Brute Force

One way of solving this problem is to just transform the words in every possible way (of course checking at each step to ensure each is a valid word), and then see if we can reach the final word.

So, for example, the word `bold` would be transformed into:

- `aold`, `bold`, . . . , `zold`
- `bald`, `bbld`, . . . , `bzld`
- `boad`, `bobd`, . . . , `bozd`
- `bola`, `bolb`, . . . , `bolz`

We will terminate (not pursue this path) if the string is not a valid word or if we've already visited this word.

This is essentially a depth-first search where there is an "edge" between two words if they are only one edit apart. This means that this algorithm will not find the shortest path. It will only find a path.

If we wanted to find the shortest path, we would want to use breadth-first search.

```
LinkedList<String> transform(String start, String stop, String[] words) {
    HashSet<String> dict = setupDictionary(words);
    HashSet<String> visited = new HashSet<String>();
    return transform(visited, start, stop, dict);
}

HashSet<String> setupDictionary(String[] words) {
    HashSet<String> hash = new HashSet<String>();
    for (String word : words) {
        hash.add(word.toLowerCase());
    }
    return hash;
}

LinkedList<String> transform(HashSet<String> visited, String startWord,
```

```
                                String stopWord, Set<String> dictionary) {
   if (startWord.equals(stopWord)) {
      LinkedList<String> path = new LinkedList<String>();
      path.add(startWord);
      return path;
   } else if (visited.contains(startWord) || !dictionary.contains(startWord)) {
      return null;
   }

   visited.add(startWord);
   ArrayList<String> words = wordsOneAway(startWord);

   for (String word : words) {
      LinkedList<String> path = transform(visited, word, stopWord, dictionary);
      if (path != null) {
         path.addFirst(startWord);
         return path;
      }
   }

   return null;
}

ArrayList<String> wordsOneAway(String word) {
   ArrayList<String> words = new ArrayList<String>();
   for (int i = 0; i < word.length(); i++) {
      for (char c = 'a'; c <= 'z'; c++) {
         String w = word.substring(0, i) + c + word.substring(i + 1);
         words.add(w);
      }
   }
   return words;
}
```

One major inefficiency in this algorithm is finding all strings that are one edit away. Right now, we're finding the strings that are one edit away and then eliminating the invalid ones.

Ideally, we want to only go to the ones that are valid.

Optimized Solution

To travel to only valid words, we clearly need a way of going from each word to a list of all the valid related words.

What makes two words "related" (one edit away)? They are one edit away if all but one character is the same. For example, `ball` and `bill` are one edit away, because they are both in the form `b_ll`. Therefore, one approach is to group all words that look like `b_ll` together.

We can do this for the whole dictionary by creating a mapping from a "wildcard word" (like `b_ll`) to a list of all words in this form. For example, for a very small dictionary like `{all, ill, ail, ape, ale}` the mapping might look like this:

```
_il -> ail
_le -> ale
_ll -> all, ill
_pe -> ape
a_e -> ape, ale
a_l -> all, ail
```

```
i_l -> ill
ai_ -> ail
al_ -> all, ale
ap_ -> ape
il_ -> ill
```

Now, when we want to know the words that are one edit away from a word like `ale`, we look up `_le`, `a_e`, and `al_` in the hash table.

The algorithm is otherwise essentially the same.

```
1  LinkedList<String> transform(String start, String stop, String[] words) {
2    HashMapList<String, String> wildcardToWordList = createWildcardToWordMap(words);
3    HashSet<String> visited = new HashSet<String>();
4    return transform(visited, start, stop, wildcardToWordList);
5  }
6
7  /* Do a depth-first search from startWord to stopWord, traveling through each word
8   * that is one edit away. */
9  LinkedList<String> transform(HashSet<String> visited, String start, String stop,
10                               HashMapList<String, String> wildcardToWordList) {
11   if (start.equals(stop)) {
12     LinkedList<String> path = new LinkedList<String>();
13     path.add(start);
14     return path;
15   } else if (visited.contains(start)) {
16     return null;
17   }
18
19   visited.add(start);
20   ArrayList<String> words = getValidLinkedWords(start, wildcardToWordList);
21
22   for (String word : words) {
23     LinkedList<String> path = transform(visited, word, stop, wildcardToWordList);
24     if (path != null) {
25       path.addFirst(start);
26       return path;
27     }
28   }
29
30   return null;
31 }
32
33 /* Insert words in dictionary into mapping from wildcard form -> word. */
34 HashMapList<String, String> createWildcardToWordMap(String[] words) {
35   HashMapList<String, String> wildcardToWords = new HashMapList<String, String>();
36   for (String word : words) {
37     ArrayList<String> linked = getWildcardRoots(word);
38     for (String linkedWord : linked) {
39       wildcardToWords.put(linkedWord, word);
40     }
41   }
42   return wildcardToWords;
43 }
44
45 /* Get list of wildcards associated with word. */
46 ArrayList<String> getWildcardRoots(String w) {
47   ArrayList<String> words = new ArrayList<String>();
```

```
48    for (int i = 0; i < w.length(); i++) {
49       String word = w.substring(0, i) + "_" + w.substring(i + 1);
50       words.add(word);
51    }
52    return words;
53 }
54
55 /* Return words that are one edit away. */
56 ArrayList<String> getValidLinkedWords(String word,
57       HashMapList<String, String> wildcardToWords) {
58    ArrayList<String> wildcards = getWildcardRoots(word);
59    ArrayList<String> linkedWords = new ArrayList<String>();
60    for (String wildcard : wildcards) {
61       ArrayList<String> words = wildcardToWords.get(wildcard);
62       for (String linkedWord : words) {
63          if (!linkedWord.equals(word)) {
64             linkedWords.add(linkedWord);
65          }
66       }
67    }
68    return linkedWords;
69 }
70
71 /* HashMapList<String, String> is a HashMap that maps from Strings to
72  * ArrayList<String>. See appendix for implementation. */
```

This will work, but we can still make it faster.

One optimization is to switch from depth-first search to breadth-first search. If there are zero paths or one path, the algorithms are equivalent speeds. However, if there are multiple paths, breadth-first search may run faster.

Breadth-first search finds the shortest path between two nodes, whereas depth-first search finds any path. This means that depth-first search might take a very long, windy path in order to find a connection when, in fact, the nodes were quite close.

Optimal Solution

As noted earlier, we can optimize this using breadth-first search. Is this as fast as we can make it? Not quite.

Imagine that the path between two nodes has length 4. With breadth-first search, we will visit about 15^4 nodes to find them.

Breadth-first search spans out very quickly.

Instead, what if we searched out from the source and destination nodes simultaneously? In this case, the breadth-first searches would collide after each had done about two levels each.

- Nodes travelled to from source: 15^2
- Nodes travelled to from destination: 15^2
- Total nodes: $15^2 + 15^2$

This is much better than the traditional breadth-first search.

We will need to track the path that we've travelled at each node.

To implement this approach, we've used an additional class BFSData. BFSData helps us keep things a bit clearer, and allows us to keep a similar framework for the two simultaneous breadth-first searches. The alternative is to keep passing around a bunch of separate variables.

```
LinkedList<String> transform(String startWord, String stopWord, String[] words) {
  HashMapList<String, String> wildcardToWordList = getWildcardToWordList(words);

  BFSData sourceData = new BFSData(startWord);
  BFSData destData = new BFSData(stopWord);

  while (!sourceData.isFinished() && !destData.isFinished()) {
    /* Search out from source. */
    String collision = searchLevel(wildcardToWordList, sourceData, destData);
    if (collision != null) {
      return mergePaths(sourceData, destData, collision);
    }

    /* Search out from destination. */
    collision = searchLevel(wildcardToWordList, destData, sourceData);
    if (collision != null) {
      return mergePaths(sourceData, destData, collision);
    }
  }

  return null;
}

/* Search one level and return collision, if any. */
String searchLevel(HashMapList<String, String> wildcardToWordList,
                   BFSData primary, BFSData secondary) {
  /* We only want to search one level at a time. Count how many nodes are
   * currently in the primary's level and only do that many nodes. We'll continue
   * to add nodes to the end. */
  int count = primary.toVisit.size();
  for (int i = 0; i < count; i++) {
    /* Pull out first node. */
    PathNode pathNode = primary.toVisit.poll();
    String word = pathNode.getWord();

    /* Check if it's already been visited. */
    if (secondary.visited.containsKey(word)) {
      return pathNode.getWord();
    }

    /* Add friends to queue. */
    ArrayList<String> words = getValidLinkedWords(word, wildcardToWordList);
    for (String w : words) {
      if (!primary.visited.containsKey(w)) {
        PathNode next = new PathNode(w, pathNode);
        primary.visited.put(w, next);
        primary.toVisit.add(next);
      }
    }
  }
  return null;
}
```

```
LinkedList<String> mergePaths(BFSData bfs1, BFSData bfs2, String connection) {
  PathNode end1 = bfs1.visited.get(connection); // end1 -> source
  PathNode end2 = bfs2.visited.get(connection); // end2 -> dest
  LinkedList<String> pathOne = end1.collapse(false); // forward
  LinkedList<String> pathTwo = end2.collapse(true); // reverse
  pathTwo.removeFirst(); // remove connection
  pathOne.addAll(pathTwo); // add second path
  return pathOne;
}

/* Methods getWildcardRoots, getWildcardToWordList, and getValidLinkedWords are
 * the same as in the earlier solution. */

public class BFSData {
  public Queue<PathNode> toVisit = new LinkedList<PathNode>();
  public HashMap<String, PathNode> visited = new HashMap<String, PathNode>();

  public BFSData(String root) {
    PathNode sourcePath = new PathNode(root, null);
    toVisit.add(sourcePath);
    visited.put(root, sourcePath);
  }

  public boolean isFinished() {
    return toVisit.isEmpty();
  }
}

public class PathNode {
  private String word = null;
  private PathNode previousNode = null;
  public PathNode(String word, PathNode previous) {
    this.word = word;
    previousNode = previous;
  }

  public String getWord() {
    return word;
  }

  /* Traverse path and return linked list of nodes. */
  public LinkedList<String> collapse(boolean startsWithRoot) {
    LinkedList<String> path = new LinkedList<String>();
    PathNode node = this;
    while (node != null) {
      if (startsWithRoot) {
        path.addLast(node.word);
      } else {
        path.addFirst(node.word);
      }
      node = node.previousNode;
    }
    return path;
  }
}
```

```
110 /* HashMapList<String, Integer> is a HashMap that maps from Strings to
111  * ArrayList<Integer>. See appendix for implementation. */
```

This algorithm's runtime is a bit harder to describe since it depends on what the language looks like, as well as the actual source and destination words. One way of expressing it is that if each word has E words that are one edit away and the source and destination are distance D, the runtime is $O(E^{D/2})$. This is how much work each breadth-first search does.

Of course, this is a lot of code to implement in an interview. It just wouldn't be possible. More realistically, you'd leave out a lot of the details. You might write just the skeleton code of `transform` and `searchLevel`, but leave out the rest.

17.23 Max Square Matrix: Imagine you have a square matrix, where each cell (pixel) is either black or white. Design an algorithm to find the maximum subsquare such that all four borders are filled with black pixels.

pg 190

SOLUTION

Like many problems, there's an easy way and a hard way to solve this. We'll go through both solutions.

The "Simple" Solution: $O(N^4)$

We know that the biggest possible square has a length of size N, and there is only one possible square of size NxN. We can easily check for that square and return if we find it.

If we do not find a square of size NxN, we can try the next best thing: `(N-1) x (N-1)`. We iterate through all squares of this size and return the first one we find. We then do the same for `N-2`, `N-3`, and so on. Since we are searching progressively smaller squares, we know that the first square we find is the biggest.

Our code works as follows:

```
1   Subsquare findSquare(int[][] matrix) {
2     for (int i = matrix.length; i >= 1; i--) {
3       Subsquare square = findSquareWithSize(matrix, i);
4       if (square != null) return square;
5     }
6     return null;
7   }
8
9   Subsquare findSquareWithSize(int[][] matrix, int squareSize) {
10    /* On an edge of length N, there are (N - sz + 1) squares of length sz. */
11    int count = matrix.length - squareSize + 1;
12
13    /* Iterate through all squares with side length squareSize. */
14    for (int row = 0; row < count; row++) {
15      for (int col = 0; col < count; col++) {
16        if (isSquare(matrix, row, col, squareSize)) {
17          return new Subsquare(row, col, squareSize);
18        }
19      }
20    }
21    return null;
22  }
23
24  boolean isSquare(int[][] matrix, int row, int col, int size) {
```

```
25    // Check top and bottom border.
26    for (int j = 0; j < size; j++){
27       if (matrix[row][col+j] == 1) {
28          return false;
29       }
30       if (matrix[row+size-1][col+j] == 1){
31          return false;
32       }
33    }
34
35    // Check left and right border.
36    for (int i = 1; i < size - 1; i++){
37       if (matrix[row+i][col] == 1){
38          return false;
39       }
40       if (matrix[row+i][col+size-1] == 1) {
41          return false;
42       }
43    }
44    return true;
45 }
```

Pre-Processing Solution: O(N³)

A large part of the slowness of the "simple" solution above is due to the fact we have to do O(N) work each time we want to check a potential square. By doing some pre-processing, we can cut down the time of `isSquare` to O(1). The time of the whole algorithm is reduced to $O(N^3)$.

If we analyze what `isSquare` does, we realize that all it ever needs to know is if the next `squareSize` items, on the right of as well as below particular cells, are zeros. We can pre-compute this data in a straightforward, iterative fashion.

We iterate from right to left, bottom to top. At each cell, we do the following computation:

```
if A[r][c] is white, zeros right and zeros below are 0
else A[r][c].zerosRight = A[r][c + 1].zerosRight + 1
     A[r][c].zerosBelow = A[r + 1][c].zerosBelow + 1
```

Below is an example of these values for a potential matrix.

(0s right, 0s below)

0,0	1,3	0,0
2,2	1,2	0,0
2,1	1,1	0,0

Original Matrix

W	B	W
B	B	W
B	B	W

Now, instead of iterating through O(N) elements, the `isSquare` method just needs to check `zerosRight` and `zerosBelow` for the corners.

Our code for this algorithm is below. Note that `findSquare` and `findSquareWithSize` is equivalent, other than a call to `processMatrix` and working with a new data type thereafter.

```
1  public class SquareCell {
2     public int zerosRight = 0;
```

```
3     public int zerosBelow = 0;
4     /* declaration, getters, setters */
5   }
6
7   Subsquare findSquare(int[][] matrix) {
8     SquareCell[][] processed = processSquare(matrix);
9     for (int i = matrix.length; i >= 1; i--) {
10      Subsquare square = findSquareWithSize(processed, i);
11      if (square != null) return square;
12    }
13    return null;
14  }
15
16  Subsquare findSquareWithSize(SquareCell[][] processed, int size) {
17    /* equivalent to first algorithm */
18  }
19
20  boolean isSquare(SquareCell[][] matrix, int row, int col, int sz) {
21    SquareCell topLeft = matrix[row][col];
22    SquareCell topRight = matrix[row][col + sz - 1];
23    SquareCell bottomLeft = matrix[row + sz - 1][col];
24
25    /* Check top, left, right, and bottom edges, respectively. */
26    if (topLeft.zerosRight < sz || topLeft.zerosBelow < sz ||
27        topRight.zerosBelow < sz || bottomLeft.zerosRight < sz) {
28      return false;
29    }
30    return true;
31  }
32
33   SquareCell[][] processSquare(int[][] matrix) {
34    SquareCell[][] processed =
35      new SquareCell[matrix.length][matrix.length];
36
37    for (int r = matrix.length - 1; r >= 0; r--) {
38      for (int c = matrix.length - 1; c >= 0; c--) {
39        int rightZeros = 0;
40        int belowZeros = 0;
41        // only need to process if it's a black cell
42        if (matrix[r][c] == 0) {
43          rightZeros++;
44          belowZeros++;
45          // next column over is on same row
46          if (c + 1 < matrix.length) {
47            SquareCell previous = processed[r][c + 1];
48            rightZeros += previous.zerosRight;
49          }
50          if (r + 1 < matrix.length) {
51            SquareCell previous = processed[r + 1][c];
52            belowZeros += previous.zerosBelow;
53          }
54        }
55        processed[r][c] = new SquareCell(rightZeros, belowZeros);
56      }
57    }
58    return processed;
```

```
59 }
```

17.24 Max Submatrix: Given an NxN matrix of positive and negative integers, write code to find the submatrix with the largest possible sum.

pg 190

SOLUTION

This problem can be approached in a variety of ways. We'll start with the brute force solution and then optimize the solution from there.

Brute Force Solution: $O(N^6)$

Like many "maximizing" problems, this problem has a straightforward brute force solution. This solution simply iterates through all possible submatrices, computes the sum, and finds the largest.

To iterate through all possible submatrices (with no duplicates), we simply need to iterate through all ordered pairs of rows, and then all ordered pairs of columns.

This solution is $O(N^6)$, since we iterate through $O(N^4)$ submatrices and it takes $O(N^2)$ time to compute the area of each.

```
1  SubMatrix getMaxMatrix(int[][] matrix) {
2    int rowCount = matrix.length;
3    int columnCount = matrix[0].length;
4    SubMatrix best = null;
5    for (int row1 = 0; row1 < rowCount; row1++) {
6      for (int row2 = row1; row2 < rowCount; row2++) {
7        for (int col1 = 0; col1 < columnCount; col1++) {
8          for (int col2 = col1; col2 < columnCount; col2++) {
9            int sum = sum(matrix, row1, col1, row2, col2);
10           if (best == null || best.getSum() < sum) {
11             best = new SubMatrix(row1, col1, row2, col2, sum);
12           }
13         }
14       }
15     }
16   }
17   return best;
18 }
19
20 int sum(int[][] matrix, int row1, int col1, int row2, int col2) {
21   int sum = 0;
22   for (int r = row1; r <= row2; r++) {
23     for (int c = col1; c <= col2; c++) {
24       sum += matrix[r][c];
25     }
26   }
27   return sum;
28 }
29
30 public class SubMatrix {
31   private int row1, row2, col1, col2, sum;
32   public SubMatrix(int r1, int c1, int r2, int c2, int sm) {
33     row1 = r1;
34     col1 = c1;
```

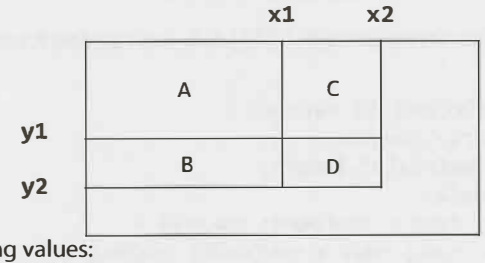
```
35        row2 = r2;
36        col2 = c2;
37        sum = sm;
38     }
39
40     public int getSum() {
41        return sum;
42     }
43  }
```

It is good practice to pull the `sum` code into its own function since it's a fairly distinct set of code.

Dynamic Programming Solution: $O(N^4)$

Notice that the earlier solution is made slower by a factor of $O(N^2)$ simply because computing the sum of a matrix is so slow. Can we reduce the time to compute the area? Yes! In fact, we can reduce the time of `computeSum` to $O(1)$.

Consider the following rectangle:



Suppose we knew the following values:

```
ValD = area(point(0, 0) -> point(x2, y2))
ValC = area(point(0, 0) -> point(x2, y1))
ValB = area(point(0, 0) -> point(x1, y2))
ValA = area(point(0, 0) -> point(x1, y1))
```

Each `Val*` starts at the origin and ends at the bottom right corner of a subrectangle.

With these values, we know the following:

```
area(D) = ValD - area(A union C) - area(A union B) + area(A).
```

Or, written another way:

```
area(D) = ValD - ValB - ValC + ValA
```

We can efficiently compute these values for all points in the matrix by using similar logic:

```
Val(x, y) = Val(x-1, y) + Val(y-1, x) - Val(x-1, y-1) + M[x][y]
```

We can precompute all such values and then efficiently find the maximum submatrix.

The following code implements this algorithm.

```
1  SubMatrix getMaxMatrix(int[][] matrix) {
2     SubMatrix best = null;
3     int rowCount = matrix.length;
4     int columnCount = matrix[0].length;
5     int[][] sumThrough = precomputeSums(matrix);
6
7     for (int row1 = 0; row1 < rowCount; row1++) {
8        for (int row2 = row1; row2 < rowCount; row2++) {
9           for (int col1 = 0; col1 < columnCount; col1++) {
10             for (int col2 = col1; col2 < columnCount; col2++) {
```

```
            int sum = sum(sumThrough, row1, col1, row2, col2);
            if (best == null || best.getSum() < sum) {
               best = new SubMatrix(row1, col1, row2, col2, sum);
            }
         }
      }
   }
   return best;
}

int[][] precomputeSums(int[][] matrix) {
   int[][] sumThrough = new int[matrix.length][matrix[0].length];
   for (int r = 0; r < matrix.length; r++) {
      for (int c = 0; c < matrix[0].length; c++) {
         int left = c > 0 ? sumThrough[r][c - 1] : 0;
         int top = r > 0 ? sumThrough[r - 1][c] : 0;
         int overlap = r > 0 && c > 0 ? sumThrough[r-1][c-1] : 0;
         sumThrough[r][c] = left + top - overlap + matrix[r][c];
      }
   }
   return sumThrough;
}

int sum(int[][] sumThrough, int r1, int c1, int r2, int c2) {
   int topAndLeft = r1 > 0 && c1 > 0 ? sumThrough[r1-1][c1-1] : 0;
   int left = c1 > 0 ? sumThrough[r2][c1 - 1] : 0;
   int top = r1 > 0 ? sumThrough[r1 - 1][c2] : 0;
   int full = sumThrough[r2][c2];
   return full - left - top + topAndLeft;
}
```

This algorithm takes $O(N^4)$ time, since it goes through each pair of rows and each pair of columns.

Optimized Solution: $O(N^3)$

Believe it or not, an even more optimal solution exists. If we have R rows and C columns, we can solve it in $O(R^2C)$ time.

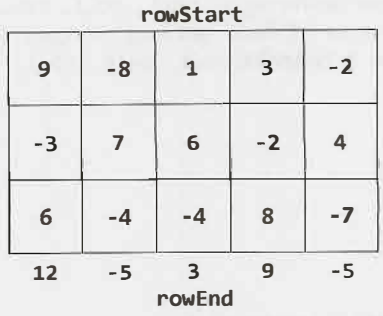
Recall the solution to the maximum subarray problem: "Given an array of integers, find the subarray with the largest sum." We can find the maximum subarray in $O(N)$ time. We will leverage this solution for this problem.

Every submatrix can be represented by a contiguous sequence of rows and a contiguous sequence of columns. If we were to iterate through every contiguous sequence of rows, we would then just need to find, for each of those, the set of columns that gives us the highest sum. That is:

```
maxSum = 0
foreach rowStart in rows
   foreach rowEnd in rows
      /* We have many possible submatrices with rowStart and rowEnd as the top and
       * bottom edges of the matrix. Find the colStart and colEnd edges that give
       * the highest sum. */
      maxSum = max(runningMaxSum, maxSum)
return maxSum
```

Now the question is, how do we efficiently find the "best" `colStart` and `colEnd`?

Picture a submatrix:



Given a `rowStart` and `rowEnd`, we want to find the `colStart` and `colEnd` that give us the highest possible sum. To do this, we can sum up each column and then apply the `maximumSubArray` function explained at the beginning of this problem.

For the earlier example, the maximum subarray is the first through fourth columns. This means that the maximum submatrix is (`rowStart`, `first column`) through (`rowEnd`, `fourth column`).

We now have pseudocode that looks like the following.

```
1  maxSum = 0
2  foreach rowStart in rows
3    foreach rowEnd in rows
4      foreach col in columns
5        partialSum[col] = sum of matrix[rowStart, col] through matrix[rowEnd, col]
6      runningMaxSum = maxSubArray(partialSum)
7      maxSum = max(runningMaxSum, maxSum)
8  return maxSum
```

The sum in lines 5 and 6 takes R*C time to compute (since it iterates through `rowStart` through `rowEnd`), so this gives us a runtime of $O(R^3C)$. We're not quite done yet.

In lines 5 and 6, we're basically adding up `a[0]...a[i]` from scratch, even though in the previous iteration of the outer for loop, we already added up `a[0]...a[i-1]`. Let's cut out this duplicated effort.

```
1  maxSum = 0
2  foreach rowStart in rows
3    clear array partialSum
4    foreach rowEnd in rows
5      foreach col in columns
6        partialSum[col] += matrix[rowEnd, col]
7      runningMaxSum = maxSubArray(partialSum)
8    maxSum = max(runningMaxSum, maxSum)
9  return maxSum
```

Our full code looks like this:

```
1  SubMatrix getMaxMatrix(int[][] matrix) {
2    int rowCount = matrix.length;
3    int colCount = matrix[0].length;
4    SubMatrix best = null;
5
6    for (int rowStart = 0; rowStart < rowCount; rowStart++) {
7      int[] partialSum = new int[colCount];
8
9      for (int rowEnd = rowStart; rowEnd < rowCount; rowEnd++) {
10       /* Add values at row rowEnd. */
11       for (int i = 0; i < colCount; i++) {
```

```
            partialSum[i] += matrix[rowEnd][i];
         }

         Range bestRange = maxSubArray(partialSum, colCount);
         if (best == null || best.getSum() < bestRange.sum) {
            best = new SubMatrix(rowStart, bestRange.start, rowEnd,
                                 bestRange.end, bestRange.sum);
         }
      }
   }
   return best;
}

Range maxSubArray(int[] array, int N) {
   Range best = null;
   int start = 0;
   int sum = 0;

   for (int i = 0; i < N; i++) {
      sum += array[i];
      if (best == null || sum > best.sum) {
         best = new Range(start, i, sum);
      }

      /* If running_sum is < 0 no point in trying to continue the series. Reset. */
      if (sum < 0) {
         start = i + 1;
         sum = 0;
      }
   }
   return best;
}

public class Range {
   public int start, end, sum;
   public Range(int start, int end, int sum) {
      this.start = start;
      this.end = end;
      this.sum = sum;
   }
}
```

This was an extremely complex problem. You would not be expected to figure out this entire problem in an interview without a lot of help from your interviewer.

17.25 Word Rectangle: Given a list of millions of words, design an algorithm to create the largest possible rectangle of letters such that every row forms a word (reading left to right) and every column forms a word (reading top to bottom). The words need not be chosen consecutively from the list, but all rows must be the same length and all columns must be the same height.

pg 190

SOLUTION

Many problems involving a dictionary can be solved by doing some pre-processing. Where can we do pre-processing?

Well, if we're going to create a rectangle of words, we know that each row must be the same length and each column must be the same length. So let's group the words of the dictionary based on their sizes. Let's call this grouping D, where `D[i]` contains the list of words of length `i`.

Next, observe that we're looking for the largest rectangle. What is the largest rectangle that could be formed? It's `length(largest word)`^{2}.

```
1  int maxRectangle = longestWord * longestWord;
2  for z = maxRectangle to 1 {
3    for each pair of numbers (i, j) where i*j = z {
4      /* attempt to make rectangle. return if successful. */
5    }
6  }
```

By iterating from the biggest possible rectangle to the smallest, we ensure that the first valid rectangle we find will be the largest possible one.

Now, for the hard part: `makeRectangle(int l, int h)`. This method attempts to build a rectangle of words which has length `l` and height h.

One way to do this is to iterate through all (ordered) sets of h words and then check if the columns are also valid words. This will work, but it's rather inefficient.

Imagine that we are trying to build a 6x5 rectangle and the first few rows are:

```
there
queen
pizza
.....
```

At this point, we know that the first column starts with `tqp`. We know—or *should* know—that no dictionary word starts with `tqp`. Why do we bother continuing to build a rectangle when we know we'll fail to create a valid one in the end?

This leads us to a more optimal solution. We can build a trie to easily look up if a substring is a prefix of a word in the dictionary. Then, when we build our rectangle, row by row, we check to see if the columns are all valid prefixes. If not, we fail immediately, rather than continue to try to build this rectangle.

The code below implements this algorithm. It is long and complex, so we will go through it step by step.

First, we do some pre-processing to group words by their lengths. We create an array of tries (one for each word length), but hold off on building the tries until we need them.

```
1  WordGroup[] groupList = WordGroup.createWordGroups(list);
2  int maxWordLength = groupList.length;
3  Trie trieList[] = new Trie[maxWordLength];
```

The `maxRectangle` method is the "main" part of our code. It starts with the biggest possible rectangle area (which is `maxWordLength`^{2}) and tries to build a rectangle of that size. If it fails, it subtracts one from the area and attempts this new, smaller size. The first rectangle that can be successfully built is guaranteed to be the biggest.

```
1  Rectangle maxRectangle() {
2    int maxSize = maxWordLength * maxWordLength;
3    for (int z = maxSize; z > 0; z--) { // start from biggest area
4      for (int i = 1; i <= maxWordLength; i ++ ) {
5        if (z % i == 0) {
6          int j = z / i;
7          if (j <= maxWordLength) {
8            /* Create rectangle of length i and height j. Note that i * j = z. */
9            Rectangle rectangle = makeRectangle(i, j);
```

```
            if (rectangle != null) return rectangle;
          }
        }
      }
    }
    return null;
  }
```

The `makeRectangle` method is called by `maxRectangle` and tries to build a rectangle of a specific *length and height*.

```
1  Rectangle makeRectangle(int length, int height) {
2    if (groupList[length-1] == null || groupList[height-1] == null) {
3      return null;
4    }
5
6    /* Create trie for word length if we haven't yet */
7    if (trieList[height - 1] == null) {
8      LinkedList<String> words = groupList[height - 1].getWords();
9      trieList[height - 1] = new Trie(words);
10   }
11
12   return makePartialRectangle(length, height, new Rectangle(length));
13 }
```

The `makePartialRectangle` method is where the action happens. It is passed in the intended, final length and height, and a partially formed rectangle. If the rectangle is already of the final height, then we just check to see if the columns form valid, complete words, and return.

Otherwise, we check to see if the columns form valid prefixes. If they do not, then we immediately break since there is no way to build a valid rectangle off of this partial one.

But, if everything is okay so far, and all the columns are valid prefixes of words, then we search through all the words of the right length, append each to the current rectangle, and recursively try to build a rectangle off of `{current rectangle with new word appended}`.

```
1  Rectangle makePartialRectangle(int l, int h, Rectangle rectangle) {
2    if (rectangle.height == h) { // Check if complete rectangle
3      if (rectangle.isComplete(l, h, groupList[h - 1])) {
4        return rectangle;
5      }
6      return null;
7    }
8
9    /* Compare columns to trie to see if potentially valid rect */
10   if (!rectangle.isPartialOK(l, trieList[h - 1])) {
11     return null;
12   }
13
14   /* Go through all words of the right length. Add each one to the current partial
15    * rectangle, and attempt to build a rectangle recursively. */
16   for (int i = 0; i < groupList[l-1].length(); i++) {
17     /* Create a new rectangle which is this rect + new word. */
18     Rectangle orgPlus = rectangle.append(groupList[l-1].getWord(i));
19
20     /* Try to build a rectangle with this new, partial rect */
21     Rectangle rect = makePartialRectangle(l, h, orgPlus);
22     if (rect != null) {
23       return rect;
```

```
24        }
25     }
26     return null;
27  }
```

The `Rectangle` class represents a partially or fully formed rectangle of words. The method `isPartialOk` can be called to check if the rectangle is, thus far, a valid one (that is, all the columns are prefixes of words). The method `isComplete` serves a similar function, but checks if each of the columns makes a full word.

```
1   public class Rectangle {
2      public int height, length;
3      public char[][] matrix;
4
5      / *Construct an "empty" rectangle. Length is fixed, but height varies as we add
6       * words. */
7      public Rectangle(int l) {
8         height = 0;
9         length = l;
10     }
11
12     / *Construct a rectangular array of letters of the specified length and height,
13      * and backed by the specified matrix of letters. (It is assumed that the length
14      * and height specified as arguments are consistent with the array argument's
15      * dimensions.) */
16     public Rectangle(int length, int height, char[][] letters) {
17        this.height = letters.length;
18        this.length = letters[0].length;
19        matrix = letters;
20     }
21
22     public char getLetter (int i, int j) { return matrix[i][j]; }
23     public String getColumn(int i) { ... }
24
25     / *Check if all columns are valid. All rows are already known to be valid since
26      * they were added directly from dictionary. */
27     public boolean isComplete(int l, int h, WordGroup groupList) {
28        if (height == h) {
29           / *Check if each column is a word in the dictionary. */
30           for (int i = 0; i < l; i++) {
31              String col = getColumn(i);
32              if (!groupList.containsWord(col)) {
33                 return false;
34              }
35           }
36           return true;
37        }
38        return false;
39     }
40
41     public boolean isPartialOK(int l, Trie trie) {
42        if (height == 0) return true;
43        for (int i = 0; i < l; i++ ) {
44           String col = getColumn(i);
45           if (!trie.contains(col)) {
46              return false;
47           }
48        }
```

```
49        return true;
50     }
51
52     / *Create a new Rectangle by taking the rows of the current rectangle and
53      * appending s. */
54     public Rectangle append(String s) { ... }
55  }
```

The WordGroup class is a simple container for all words of a specific length. For easy lookup, we store the words in a hash table as well as in an ArrayList.

The lists in WordGroup are created through a static method called createWordGroups.

```
1   public class WordGroup {
2     private HashMap<String, Boolean> lookup = new HashMap<String, Boolean>();
3     private ArrayList<String> group = new ArrayList<String>();
4     public boolean containsWord(String s) { return lookup.containsKey(s); }
5     public int length() { return group.size(); }
6     public String getWord(int i) { return group.get(i); }
7     public ArrayList<String> getWords() { return group; }
8
9     public void addWord (String s) {
10      group.add(s);
11      lookup.put(s, true);
12    }
13
14    public static WordGroup[] createWordGroups(String[] list) {
15      WordGroup[] groupList;
16      int maxWordLength = 0;
17      / *Find the length of the longest word */
18      for (int i = 0; i < list.length; i++) {
19        if (list[i].length() > maxWordLength) {
20          maxWordLength = list[i].length();
21        }
22      }
23
24      / *Group the words in the dictionary into lists of words of same length.
25       * groupList[i] will contain a list of words, each of length (i+1). */
26      groupList = new WordGroup[maxWordLength];
27      for (int i = 0; i < list.length; i++) {
28        / *We do wordLength - 1 instead of just wordLength since this is used as
29         * an index and no words are of length 0 */
30        int wordLength = list[i].length() - 1;
31        if (groupList[wordLength] == null) {
32          groupList[wordLength] = new WordGroup();
33        }
34        groupList[wordLength].addWord(list[i]);
35      }
36      return groupList;
37    }
38  }
```

The full code for this problem, including the code for Trie and TrieNode, can be found in the code attachment. Note that in a problem as complex as this, you'd most likely only need to write the pseudocode. Writing the entire code would be nearly impossible in such a short amount of time.

17.26 Sparse Similarity: The similarity of two documents (each with distinct words) is defined to be the size of the intersection divided by the size of the union. For example, if the documents consist of integers, the similarity of `{1, 5, 3}` and `{1, 7, 2, 3}` is `0.4`, because the intersection has size 2 and the union has size 5.

We have a long list of documents (with distinct values and each with an associated ID) where the similarity is believed to be "sparse." That is, any two arbitrarily selected documents are very likely to have similarity 0. Design an algorithm that returns a list of pairs of document IDs and the associated similarity.

Print only the pairs with similarity greater than 0. Empty documents should not be printed at all. For simplicity, you may assume each document is represented as an array of distinct integers.

EXAMPLE

Input:

```
13: {14, 15, 100, 9, 3}
16: {32, 1, 9, 3, 5}
19: {15, 29, 2, 6, 8, 7}
24: {7, 10}
```

Output:

```
ID1, ID2 : SIMILARITY
13, 19   : 0.1
13, 16   : 0.25
19, 24   : 0.14285714285714285
```

pg 190

SOLUTION

This sounds like quite a tricky problem, so let's start off with a brute force algorithm. If nothing else, it will help wrap our heads around the problem.

Remember that each document is an array of distinct "words", and each is just an integer.

Brute Force

A brute force algorithm is as simple as just comparing all arrays to all other arrays. At each comparison, we compute the size of the intersection and size of the union of the two arrays.

Note that we only want to print this pair if the similarity is greater than 0. The union of two arrays can never be zero (unless both arrays are empty, in which case we don't want them printed anyway). Therefore, we are really just printing the similarity if the intersection is greater than 0.

How do we compute the size of the intersection and the union?

The intersection means the number of elements in common. Therefore, we can just iterate through the first array (A) and check if each element is in the second array (B). If it is, increment an `intersection` variable.

To compute the union, we need to be sure that we don't double count elements that are in both. One way to do this is to count up all the elements in A that are *not* in B. Then, add in all the elements in B. This will avoid double counting as the duplicate elements are only counted with B.

Alternatively, we can think about it this way. If we *did* double count elements, it would mean that elements in the intersection (in both A and B) were counted twice. Therefore, the easy fix is to just remove these duplicate elements.

```
union(A, B) = A + B - intersection(A, B)
```

This means that all we really need to do is compute the intersection. We can derive the union, and therefore similarity, from that immediately.

This gives us an $O(AB)$ algorithm, just to compare two arrays (or documents).

However, we need to do this for all pairs of D documents. If we assume each document has at most W words then the runtime is $O(D^2 W^2)$.

Slightly Better Brute Force

As a quick win, we can optimize the computation for the similarity of two arrays. Specifically, we need to optimize the intersection computation.

We need to know the number of elements in common between the two arrays. We can throw all of A's elements into a hash table. Then we iterate through B, incrementing `intersection` every time we find an element in A.

This takes $O(A + B)$ time. If each array has size W and we do this for D arrays, then this takes $O(D^2 W)$.

Before implementing this, let's first think about the classes we'll need.

We'll need to return a list of document pairs and their similarities. We'll use a `DocPair` class for this. The exact return type will be a hash table that maps from `DocPair` to a double representing the similarity.

```
1  public class DocPair {
2    public int doc1, doc2;
3
4    public DocPair(int d1, int d2) {
5      doc1 = d1;
6      doc2 = d2;
7    }
8
9    @Override
10   public boolean equals(Object o) {
11     if (o instanceof DocPair) {
12       DocPair p = (DocPair) o;
13       return p.doc1 == doc1 && p.doc2 == doc2;
14     }
15     return false;
16   }
17
18   @Override
19   public int hashCode() { return (doc1 * 31) ^ doc2; }
20 }
```

It will also be useful to have a class that represents the documents.

```
1  public class Document {
2    private ArrayList<Integer> words;
3    private int docId;
4
5    public Document(int id, ArrayList<Integer> w) {
6      docId = id;
7      words = w;
8    }
9
10   public ArrayList<Integer> getWords() { return words; }
11   public int getId() { return docId; }
```

```
12    public int size() { return words == null ? 0 : words.size(); }
13 }
```

Strictly speaking, we don't need any of this. However, readability is important, and it's a lot easier to read `ArrayList<Document>` than `ArrayList<ArrayList<Integer>>`.

Doing this sort of thing not only shows good coding style, it also makes your life in an interview a lot easier. You have to write a lot less. (You probably would not define the entire `Document` class, unless you had extra time or your interviewer asked you to.)

```
1  HashMap<DocPair, Double> computeSimilarities(ArrayList<Document> documents) {
2    HashMap<DocPair, Double> similarities = new HashMap<DocPair, Double>();
3    for (int i = 0; i < documents.size(); i++) {
4      for (int j = i + 1; j < documents.size(); j++) {
5        Document doc1 = documents.get(i);
6        Document doc2 = documents.get(j);
7        double sim = computeSimilarity(doc1, doc2);
8        if (sim > 0) {
9          DocPair pair = new DocPair(doc1.getId(), doc2.getId());
10         similarities.put(pair, sim);
11       }
12     }
13   }
14   return similarities;
15 }
16
17 double computeSimilarity(Document doc1, Document doc2) {
18   int intersection = 0;
19   HashSet<Integer> set1 = new HashSet<Integer>();
20   set1.addAll(doc1.getWords());
21
22   for (int word : doc2.getWords()) {
23     if (set1.contains(word)) {
24       intersection++;
25     }
26   }
27
28   double union = doc1.size() + doc2.size() - intersection;
29   return intersection / union;
30 }
```

Observe what's happening on line 28. Why did we make `union` a `double`, when it's obviously an `integer`?

We did this to avoid an integer division bug. If we didn't do this, the division would "round" down to an integer. This would mean that the similarity would almost always return 0. Oops!

Slightly Better Brute Force (Alternate)

If the documents were sorted, you could compute the intersection between two documents by walking through them in sorted order, much like you would when doing a sorted merge of two arrays.

This would take $O(A + B)$ time. This is the same time as our current algorithm, but less space. Doing this on D documents with W words each would take $O(D^2 W)$ time.

Since we don't know that the arrays are sorted, we could first sort them. This would take $O(D * W \log W)$ time. The full runtime then is $O(D * W \log W + D^2 W)$.

We cannot necessarily assume that the second part "dominates" the first one, because it doesn't necessarily. It depends on the relative size of D and $\log W$. Therefore, we need to keep both terms in our runtime expression.

Optimized (Somewhat)

It is useful to create a larger example to really understand the problem.

```
13: {14, 15, 100, 9, 3}
16: {32, 1, 9, 3, 5}
19: {15, 29, 2, 6, 8, 7}
24: {7, 10, 3}
```

At first, we might try various techniques that allow us to more quickly eliminate potential comparisons. For example, could we compute the min and max values in each array? If we did that, then we'd know that arrays with no overlap in ranges don't need to be compared.

The problem is that this doesn't really fix our runtime issue. Our best runtime thus far is $O(D^2 W)$. With this change, we're still going to be comparing all $O(D^2)$ pairs, but the $O(W)$ part might go to $O(1)$ sometimes. That $O(D^2)$ part is going to be a really big problem when D gets large.

Therefore, let's focus on reducing that $O(D^2)$ factor. That is the "bottleneck" in our solution. Specifically, this means that, given a document docA, we want to find all documents with some similarity—and we want to do this without "talking" to each document.

What would make a document similar to docA? That is, what characteristics define the documents with similarity > 0?

Suppose docA is {14, 15, 100, 9, 3}. For a document to have similarity > 0, it needs to have a 14, a 15, a 100, a 9, or a 3. How can we quickly gather a list of all documents with one of those elements?

The slow (and, really, only way) is to read every single word from every single document to find the documents that contain a 14, a 15, a 100, a 9, or a 3. That will take $O(DW)$ time. Not good.

However, note that we're doing this repeatedly. We can reuse the work from one call to the next.

If we build a hash table that maps from a word to all documents that contain that word, we can very quickly know the documents that overlap with docA.

```
1 -> 16
2 -> 19
3 -> 13, 16, 24
5 -> 16
6 -> 19
7 -> 19, 24
8 -> 19
9 -> 13, 16
...
```

When we want to know all the documents that overlap with docA, we just look up each of docA's items in this hash table. We'll then get a list of all documents with some overlap. Now, all we have to do is compare docA to each of those documents.

If there are P pairs with similarity > 0, and each document has W words, then this will take $O(PW)$ time (plus $O(DW)$ time to create and read this hash table). Since we expect P to be much less than D^2, this is much better than before.

Optimized (Better)

Let's think about our previous algorithm. Is there any way we can make it more optimal?

If we consider the runtime—O(PW + DW)—we probably can't get rid of the O(DW) factor. We have to touch each word at least once, and there are O(DW) words. Therefore, if there's an optimization to be made, it's probably in the O(PW) term.

It would be difficult to eliminate the P part in O(PW) because we have to at least print all P pairs (which takes O(P) time). The best place to focus, then, is on the W part. Is there some way we can do less than O(W) work for each pair of similar documents?

One way to tackle this is to analyze what information the hash table gives us. Consider this list of documents:

```
12: {1, 5, 9}
13: {5, 3, 1, 8}
14: {4, 3, 2}
15: {1, 5, 9, 8}
17: {1, 6}
```

If we look up document 12's elements in a hash table for this document, we'll get:

```
1 -> {12, 13, 15, 17}
5 -> {12, 13, 15}
9 -> {12, 15}
```

This tells us that documents 13, 15, and 17 have some similarity. Under our current algorithm, we would now need to compare document 12 to documents 13, 15, and 17 to see the number of elements document 12 has in common with each (that is, the size of the intersection). The union can be computed from the document sizes and the intersection, as we did before.

Observe, though, that document 13 appeared twice in the hash table, document 15 appeared three times, and document 17 appeared once. We discarded that information. But can we use it instead? What does it indicate that some documents appeared multiple times and others didn't?

Document 13 appeared twice because it has two elements (1 and 5) in common. Document 17 appeared once because it has only one element (1) in common. Document 15 appeared three times because it has three elements (1, 5, and 9) in common. This information can actually directly give us the size of the intersection.

We could go through each document, look up the items in the hash table, and then count how many times each document appears in each item's lists. There's a more direct way to do it.

1. As before, build a hash table for a list of documents.
2. Create a new hash table that maps from a document pair to an integer (which will indicate the size of the intersection).
3. Read the first hash table by iterating through each list of documents.
4. For each list of documents, iterate through the pairs in that list. Increment the intersection count for each pair.

Comparing this runtime to the previous one is a bit tricky. One way we can look at it is to realize that before we were doing O(W) work for each similar pair. That's because once we noticed that two documents were similar, we touched every single word in each document. With this algorithm, we're only touching the words that actually overlap. The worst cases are still the same, but for many inputs this algorithm will be faster.

```
1   HashMap<DocPair, Double>
2   computeSimilarities(HashMap<Integer, Document> documents) {
```

```
  HashMapList<Integer, Integer> wordToDocs = groupWords(documents);
  HashMap<DocPair, Double> similarities = computeIntersections(wordToDocs);
  adjustToSimilarities(documents, similarities);
  return similarities;
}

/* Create hash table from each word to where it appears. */
HashMapList<Integer, Integer> groupWords(HashMap<Integer, Document> documents) {
  HashMapList<Integer, Integer> wordToDocs = new HashMapList<Integer, Integer>();

  for (Document doc : documents.values()) {
    ArrayList<Integer> words = doc.getWords();
    for (int word : words) {
      wordToDocs.put(word, doc.getId());
    }
  }

  return wordToDocs;
}

/* Compute intersections of documents. Iterate through each list of documents and
 * then each pair within that list, incrementing the intersection of each page. */
HashMap<DocPair, Double> computeIntersections(
    HashMapList<Integer, Integer> wordToDocs {
  HashMap<DocPair, Double> similarities = new HashMap<DocPair, Double>();
  Set<Integer> words = wordToDocs.keySet();
  for (int word : words) {
    ArrayList<Integer> docs = wordToDocs.get(word);
    Collections.sort(docs);
    for (int i = 0; i < docs.size(); i++) {
      for (int j = i + 1; j < docs.size(); j++) {
        increment(similarities, docs.get(i), docs.get(j));
      }
    }
  }

  return similarities;
}

/* Increment the intersection size of each document pair. */
void increment(HashMap<DocPair, Double> similarities, int doc1, int doc2) {
  DocPair pair = new DocPair(doc1, doc2);
  if (!similarities.containsKey(pair)) {
    similarities.put(pair, 1.0);
  } else {
    similarities.put(pair, similarities.get(pair) + 1);
  }
}

/* Adjust the intersection value to become the similarity. */
void adjustToSimilarities(HashMap<Integer, Document> documents,
                          HashMap<DocPair, Double> similarities) {
  for (Entry<DocPair, Double> entry : similarities.entrySet()) {
    DocPair pair = entry.getKey();
    Double intersection = entry.getValue();
    Document doc1 = documents.get(pair.doc1);
```

```
59        Document doc2 = documents.get(pair.doc2);
60        double union = (double) doc1.size() + doc2.size() - intersection;
61        entry.setValue(intersection / union);
62     }
63  }
64
65  /* HashMapList<Integer, Integer> is a HashMap that maps from Integer to
66   * ArrayList<Integer>. See appendix for implementation. */
```

For a set of documents with sparse similarity, this will run much faster than the original naive algorithm, which compares all pairs of documents directly.

Optimized (Alternative)

There's an alternative algorithm that some candidates might come up with. It's slightly slower, but still quite good.

Recall our earlier algorithm that computed the similarity between two documents by sorting them. We can extend this approach to multiple documents.

Imagine we took all of the words, tagged them by their original document, and then sorted them. The prior list of documents would look like this:

1_{12}, 1_{13}, 1_{15}, 1_{16}, 2_{14}, 3_{13}, 3_{14}, 4_{14}, 5_{12}, 5_{13}, 5_{15}, 6_{16}, 8_{13}, 8_{15}, 9_{12}, 9_{15}

Now we have essentially the same approach as before. We iterate through this list of elements. For each sequence of identical elements, we increment the intersection counts for the corresponding pair of documents.

We will use an Element class to group together documents and words. When we sort the list, we will sort first on the word but break ties on the document ID.

```
1   class Element implements Comparable<Element> {
2     public int word, document;
3     public Element(int w, int d) {
4       word = w;
5       document = d;
6     }
7
8     /* When we sort the words, this function will be used to compare the words. */
9     public int compareTo(Element e) {
10      if (word == e.word) {
11        return document - e.document;
12      }
13      return word - e.word;
14    }
15  }
16
17  HashMap<DocPair, Double> computeSimilarities(
18        HashMap<Integer, Document> documents) {
19    ArrayList<Element> elements = sortWords(documents);
20    HashMap<DocPair, Double> similarities = computeIntersections(elements);
21    adjustToSimilarities(documents, similarities);
22    return similarities;
23  }
24
25  /* Throw all words into one list, sorting by the word and then the document. */
26  ArrayList<Element> sortWords(HashMap<Integer, Document> docs) {
27    ArrayList<Element> elements = new ArrayList<Element>();
```

```
28    for (Document doc : docs.values()) {
29       ArrayList<Integer> words = doc.getWords();
30       for (int word : words) {
31          elements.add(new Element(word, doc.getId()));
32       }
33    }
34    Collections.sort(elements);
35    return elements;
36 }
37
38 /* Increment the intersection size of each document pair. */
39 void increment(HashMap<DocPair, Double> similarities, int doc1, int doc2) {
40    DocPair pair = new DocPair(doc1, doc2);
41    if (!similarities.containsKey(pair)) {
42       similarities.put(pair, 1.0);
43    } else {
44       similarities.put(pair, similarities.get(pair) + 1);
45    }
46 }
47
48 /* Adjust the intersection value to become the similarity. */
49 HashMap<DocPair, Double> computeIntersections(ArrayList<Element> elements) {
50    HashMap<DocPair, Double> similarities = new HashMap<DocPair, Double>();
51
52    for (int i = 0; i < elements.size(); i++) {
53       Element left = elements.get(i);
54       for (int j = i + 1; j < elements.size(); j++) {
55          Element right = elements.get(j);
56          if (left.word != right.word) {
57             break;
58          }
59          increment(similarities, left.document, right.document);
60       }
61    }
62    return similarities;
63 }
64
65 /* Adjust the intersection value to become the similarity. *
66 void adjustToSimilarities(HashMap<Integer, Document> documents,
67                           HashMap<DocPair, Double> similarities) {
68    for (Entry<DocPair, Double> entry : similarities.entrySet()) {
69       DocPair pair = entry.getKey();
70       Double intersection = entry.getValue();
71       Document doc1 = documents.get(pair.doc1);
72       Document doc2 = documents.get(pair.doc2);
73       double union = (double) doc1.size() + doc2.size() - intersection;
74       entry.setValue(intersection / union);
75    }
76 }
```

The first step of this algorithm is slower than that of the prior algorithm, since it has to sort rather than just add to a list. The second step is essentially equivalent.

Both will run much faster than the original naive algorithm.

Advanced Topics

XI

This section includes topics that are mostly beyond the scope of interviews but can come up on occasion. Interviewers shouldn't be surprised if you don't know these topics well. Feel free to dive into these topics if you want to. If you're pressed for time, they're low priority.

XI

Advanced Topics

When writing the 6th edition, I had a number of debates about what should and shouldn't be included. Red-black trees? Dijkstra's algorithm? Topological sort?

On one hand, I'd had a number of requests to include these topics. Some people insisted that these topics are asked "all the time" (in which case, they have a very different idea of what this phrase means!). There was clearly a desire—at least from some people—to include them. And learning more can't hurt, right?

On the other hand, I know these topics to be rarely asked. It happens, of course. Interviewers are individuals and might have their own ideas of what is "fair game" or "relevant" for an interview. But it's rare. When it does come up, if you don't know the topic, it's unlikely to be a big red flag.

> Admittedly, as an interviewer, I *have* asked candidates questions where the solution was essentially an application of one of these algorithms. On the rare occasions that a candidate already knew the algorithm, they did not benefit from this knowledge (nor were they hurt by it). I want to evaluate your ability to solve a problem you haven't seen before. So, I'll take into account whether you know the underlying algorithm in advance.

I believe in giving people a fair expectation of the interview, not scaring people into excess studying. I also have no interest in making the book more "advanced" so as to help book sales, at the expense of your time and energy. That's not fair or right to do to you.

(Additionally, I didn't want to give interviewers—who I know to be reading this—the impression that they can or should be covering these more advanced topics. Interviewers: If you ask about these topics, you're testing knowledge of algorithms. You're just going to wind up eliminating a lot of perfectly smart people.)

But there are many borderline "important" topics. They're not often asked, but sometimes they are.

Ultimately, I decided to leave the decision in your hands. After all, you know better than I do how thorough you want to be in your preparation. If you want to do an extra thorough job, read this. If you just love learning data structures and algorithms, read this. If you want to see new ways of approaching problems, read this.

But if you're pressed for time, this studying isn't a super high priority.

Useful Math

Here's some math that can be useful in some questions. There are more formal proofs that you can look up online, but we'll focus here on giving you the intuition behind them. You can think of these as informal proofs.